Hardware/Software Partitioning of MLPerf Tiny on a RISC-V SystemC Platform

Step-by-step guide: setup, code, runs and verification · COE 407/661 System on Chip term project · Abdelrahman Youssef · package of 28 October 2026 (with step 5)

This guide takes you from an empty container to the final figures. Every step says **what it does**, **what to type**, **what you should see**, and **what to do if it fails**. All numbers printed in green boxes come from the run made when this package was tested; the simulation is deterministic, so your cycle counts should match them exactly.

1. The project in one page
2. What is in the package
3. Setup on your machine (once)
4. Step A: test the accelerator alone (SystemC 2.3.3)
5. Step B: first program on the platform
6. Step 0: reproduce the anchor paper's comparison
7. Step 1: software only
8. Step 2: a simple accelerator
9. Step 3: a better accelerator
10. Step 4: design-space exploration and figures
11. Step 5: DMA and bus design (extension)
12. How the code works
13. Verification checklist
14. Troubleshooting
15. Limits of the model (say these in the paper)
16. References

1. The project in one page

Goal. Run the convolution, depthwise and fully-connected layers of the three MLPerf Tiny models on a RISC-V CPU inside a SystemC simulation, first in software, then with a simple accelerator, then with a better one, and explain where the time goes at each step.

Why SystemC. STONNE (the anchor paper) tells you how many cycles an accelerator needs for one layer. It does not tell you how long the *system* needs: the CPU preparing the data, the bus moving it, the driver waiting for the result, and the layers the accelerator cannot run. The SystemC platform adds exactly those costs, so the measurements are end-to-end.

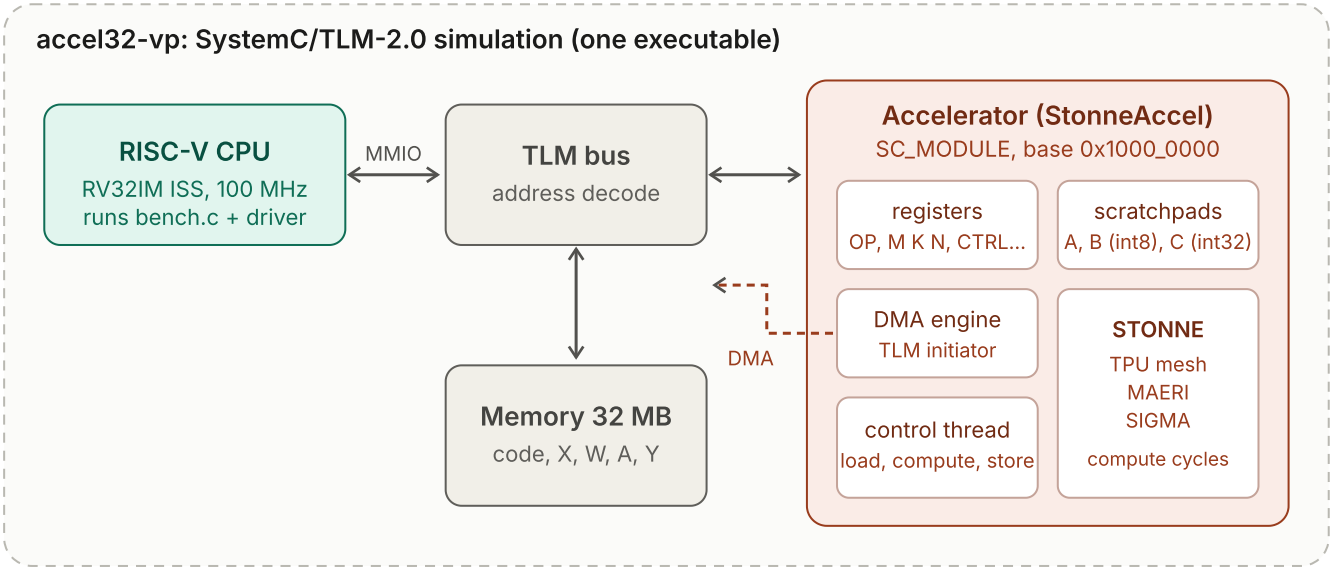
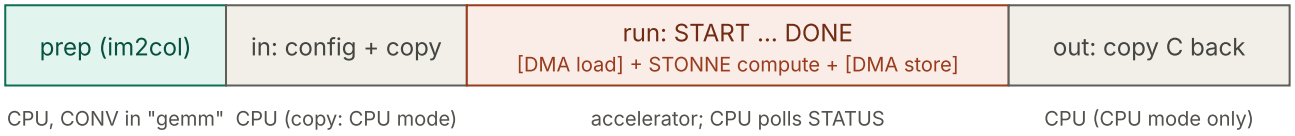


Figure 1. The simulated SoC (accel32-vp). The CPU, bus and memory come from RISC-V VP++; the accelerator (orange) is the SystemC module written for this project. Its compute time comes from STONNE.

Step	What runs on the CPU	What runs on the accelerator	Script
0	STONNE alone: TPU vs MAERI vs SIGMA on single layers (no CPU, no SystemC)		`step0_reproduce_stonne.sh`
A	the accelerator model alone, driven by a SystemC testbench on the course SystemC 2.3.3		`unit_test/make test`
1	everything	nothing	`step1_software.sh`
2	im2col, depthwise layers, data copies	GEMMs on a rigid 16×16 TPU-like mesh	`step2_simple_accel.sh`
3a	same as step 2	GEMMs on the flexible MAERI fabric	`step3_better_accel.sh`
3b	only the driver	CONV and depthwise natively (no im2col), PW and FC as GEMMs	
4	sweeps: mesh size, distribution bandwidth, DMA width, pruned weights		`dse.py`, `plot_results.py`
5	only the driver; sleeps or does other work while it waits	as 3b, plus tiled double-buffered DMA, a completion interrupt, and a shared bus with burst / cycle-stealing / transparent DMA	`unit_test/make test5`, `step5_dma_extension.sh`, `dse5.py`

What is measured. For every layer and every mode the program prints one CSV line with the cycles spent in each phase (Figure 2): preparing the data on the CPU (im2col), moving the inputs in, running the job (START until DONE), and moving the output back. CPU-copy mode means the CPU writes the operands into the accelerator's scratchpads word by word; DMA mode means the accelerator fetches them itself.



total = prep + in + run + out, measured with the platform timer; depthwise layers left on the CPU count as "CPU-only".

Figure 2. The phases of one offloaded layer, as measured by the driver with the platform timer. In DMA mode the DMA load and store happen inside "run".

2. What is in the package

```
soc-accel-project/
  README.md                 short orientation
  docs/                     Guide.pdf (this file), Proposal (.docx/.pdf)
  setup/                    01..05 setup scripts, env.sh (paths for every shell)
  accel/                    the accelerator (shared by the unit test and the platform)
    accel_regs.h              register map, also used by the C driver
    stonne_accel.h            SystemC/TLM-2.0 module: registers, scratchpads, DMA, control thread
    stonne_backend.h/.cpp     the only file that talks to STONNE (GEMM and CONV jobs)
    shared_bus.h              step 5: the shared, arbitrated bus (CPU port + DMA reservations)
  unit_test/                tb_accel.cpp (step A) and tb_dma.cpp (step 5) + Makefile, course SystemC
2.3.3
  vp_platform/accel/        main.cpp + CMakeLists.txt: the accel32-vp platform for RISC-V VP++
  sw/                       bare-metal C for the RISC-V CPU
    bench.c                   runs every layer in the chosen modes, prints CSV
    kernels.c                 software convolution, depthwise, FC, im2col
    accel_driver.c            driver: CPU-copy and DMA, GEMM and CONV jobs (unchanged in step 5)
    accel_ext.c, bench_dma.c  step 5: tiles, double buffering, poll/WFI/background wait; its
benchmark
    trap.S, irq.c             step 5: trap entry, PLIC set-up, interrupt handler, race-free WFI
    layers.h                  the 48 MLPerf Tiny layers (generated)
    runtime.c, bootstrap.S    printing, timer, start-up code
  workloads/                extract_layers.py + layers_mlperf_tiny.csv
  scripts/                  step0..step3 and step5 scripts, run_vp.sh, summarize.py, summarize5.py,
                            dse.py, dse5.py, plot_results.py, plot_step5.py
  patches/stonne-accel.patch  one line added to STONNE (exposes the cycle count)
  reference_results/        the CSV, log and figure files of the tested run
  results/                  your runs write here
```

Third-party code is **not** in the zip. The setup scripts download it at the exact commits that were tested: STONNE `22634b8` (June 2025), RISC-V VP++ `9156184` (October 2026) and MLPerf Tiny `4addd0f` (July 2026).

3. Setup on your machine (once, about 30–45 minutes)

You use the same Docker image as in the course (`systemc-env`: Ubuntu 24.04, SystemC 2.3.3 built at C++20 in `/opt/systemc`). Nothing new is installed on Windows.

3.1 Unzip and start a container

Extract `soc-accel-project.zip` into `C:\kfupm\third semester\soc\Docker\systemc_work`, so that the folder `systemc_work\soc-accel-project\setup` exists. Then, in a **new** PowerShell window:

PowerShell (Windows)
```
docker run -it --name soc-accel -v "C:\kfupm\third semester\soc\Docker\systemc_work:/workspace"
systemc-env bash
```

container (bash)
```
cd /workspace/soc-accel-project
ls                       # README.md accel docs patches reference_results results scripts setup sw
unit_test vp_platform workloads
echo $SYSTEMC_HOME       # /opt/systemc
```

Later sessions: `docker start -ai soc-accel`. A second shell: `docker exec -it soc-accel bash`.

> **Why some things live outside /workspace**
>
> `/workspace` is your Windows folder. Builds there are slow and Windows does not keep Linux execute permissions, so the setup scripts put the downloaded tools (STONNE, VP++, MLPerf Tiny) and their builds in `/opt/soc-ext` inside the container. Your own code and results stay in `/workspace`. That is why step 3.6 saves the container as an image: otherwise the builds are lost if you delete the container. Always run scripts as `bash script.sh` (the Windows folder loses the execute bit).

3.2 Install the packages

What it does: installs git, CMake, the Boost libraries VP++ needs, the RISC-V cross-compiler (`gcc-riscv64-unknown-elf`, which also builds 32-bit code), Python with matplotlib, and optionally the `tflite` reader.

container (bash)
```
bash setup/01_install_packages.sh
```

you should see
```
g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
cmake version 3.28.3
riscv64-unknown-elf-gcc (13.2.0-11ubuntu1+12) 13.2.0
/usr/lib/gcc/riscv64-unknown-elf/13.2.0/rv32im/ilp32/libgcc.a
/opt/systemc/lib/libsystemc.so
SystemC 2.3.3 found in /opt/systemc
01_install_packages: OK
```

The `rv32im/ilp32/libgcc.a` line proves the compiler can build for the 32-bit CPU used here.

3.3 Download the sources

What it does: fetches STONNE, RISC-V VP++ (plus its vendored SystemC and one parser library) and the MLPerf Tiny model files into `/opt/soc-ext`, each at the tested commit, and applies the one-line STONNE patch (a public `getNCycles()` getter, because STONNE keeps its cycle counter private).

container (bash)
```
bash setup/02_get_sources.sh
```

you should see
```
/opt/soc-ext/stonne at 22634b8
STONNE patched (getNCycles)
/opt/soc-ext/riscv-vp-plusplus at 9156184
...
02_get_sources: OK
```

3.4 Build STONNE and check it

What it does: builds STONNE's own command-line simulator and runs the first ResNet-8 layer (as a 1024×27×16 GEMM) on each of the three accelerators. STONNE compares every result with a CPU GEMM.

container (bash)
```
bash setup/03_build_stonne.sh
```

you should see
```
ResNet-8 first conv as a GEMM (M=1024, K=27, N=16), 256 multipliers:
  TPU 16x16  : 3968 cycles, result PASS
  MAERI bw256: 2061 cycles, result PASS
  SIGMA bw256: 4118 cycles, result PASS
03_build_stonne: OK
```

3.5 Build the RISC-V platform with the accelerator

What it does: copies `vp_platform/accel` into VP++ as a new platform, configures VP++ with the paths of `accel/` and STONNE, and builds only `accel32-vp`. The first build also compiles VP++'s own SystemC 3.0.

container (bash)
```
bash setup/04_build_vp.sh
```

you should see
```
built /opt/soc-ext/riscv-vp-plusplus/vp/build/bin/accel32-vp
        SystemC 3.0.1-Accellera --- (date of your build)   (banner of VP++'s own SystemC)
  --accel-arch arg          accelerator: tpu | maeri | sigma (default tpu)
04_build_vp: OK
```

The tested build took 4 minutes on 2 cores; on your 28-thread laptop expect 1–2 minutes.

> **Two SystemC versions, one accelerator source**
>
> VP++ brings its own SystemC (3.0, C++17) and does not use `/opt/systemc`. The accelerator files in `accel/` use only standard SystemC and TLM-2.0, so the same source compiles into the platform (SystemC 3.0) and into the step-A unit test (your course SystemC 2.3.3, C++20). That is shown in step A and is worth one sentence in the paper.

3.6 Load the paths in every shell and save the container

container (bash)
```
bash setup/05_save_env.sh
source setup/env.sh      # for the current shell
```

PowerShell (Windows)
```
docker commit soc-accel soc-accel-ready
```

From now on you can start fresh containers from `soc-accel-ready` with everything built: `docker run -it --name soc-accel2 -v "C:\...\systemc_work:/workspace" soc-accel-ready bash`.

4. Step A: test the accelerator alone (course SystemC 2.3.3)

What it does. Builds `tb_accel`: a SystemC testbench in which a driver module (standing in for the CPU) talks to the accelerator over TLM-2.0, and a memory module serves the accelerator's DMA. For each job the driver writes the registers, moves the data (CPU-copy or DMA), starts the job, polls `STATUS`, and compares the result with plain C++ code. It runs GEMM jobs on all three architectures and CONV jobs (standard, strided, padded and depthwise) on MAERI, checks that TPU and SIGMA refuse CONV with the ERROR flag, and checks that a job too big for the scratchpads is refused.

container (bash)

```
cd unit_test
make -j"$(nproc)"     # first time also compiles STONNE (about 1 minute)
make test
cd ..
```

you should see

```
[accel] TPU, 256 PEs, dn_bw 32, DMA 4 B/cycle, burst 64 B + 10 cycles
  …
resnet8_conv0 CPU GEMM 1024x27x16       | in   7024 run   3973 out  16384 | ld      0 cmp   3968 st      0 |  43.5% PASS
resnet8_conv0 DMA GEMM 1024x27x16       | in      7 run  42011 out      0 | ld  11410 cmp   3968 st  26624 |  43.5% PASS
dscnn_pw2     CPU GEMM 125x64x64        | in   3028 run   3159 out   8000 | ld      0 cmp   3156 st      0 |  63.4% PASS
dscnn_pw2     DMA GEMM 125x64x64        | in      7 run  21078 out      0 | ld   4914 cmp   3156 st  13000 |  63.4% PASS
dscnn_pw2_50z CPU GEMM 125x64x64 w0=50% | in   3028 run   3159 out   8000 | ld      0 cmp   3156 st      0 |  63.4% PASS
dscnn_pw2_50z DMA GEMM 125x64x64 w0=50% | in      7 run  21078 out      0 | ld   4914 cmp   3156 st  13000 |  63.4% PASS
  …
CAPS register: GEMM yes, CONV no
  …
conv3x3       CPU CONV on this architecture -> ERROR flag set (expected)
  …
conv3x3       DMA CONV on this architecture -> ERROR flag set (expected)
  …
ALL TESTS PASSED (0 failures), simulated time 1410490 ns
[accel] MAERI, 256 PEs, dn_bw 256, DMA 4 B/cycle, burst 64 B + 10 cycles
ID register: 0x53544e45 (ok)
tiny          CPU GEMM 4x16x4           | in     36 run     24 out     16 | ld      0 cmp     13 st      0 |   7.7% PASS
tiny          DMA GEMM 4x16x4           | in      7 run    101 out      0 | ld     52 cmp     13 st     26 |   7.7% PASS
resnet8_conv0 CPU GEMM 1024x27x16       | in   7024 run   2070 out  16384 | ld      0 cmp   2061 st      0 |  83.8% PASS
resnet8_conv0 DMA GEMM 1024x27x16       | in      7 run  40097 out      0 | ld  11410 cmp   2061 st  26624 |  83.8% PASS
dscnn_pw2     CPU GEMM 125x64x64        | in   3028 run   2037 out   8000 | ld      0 cmp   2027 st      0 |  98.7% PASS
dscnn_pw2     DMA GEMM 125x64x64        | in      7 run  19945 out      0 | ld   4914 cmp   2027 st  13000 |  98.7% PASS
dscnn_pw2_50z CPU GEMM 125x64x64 w0=50% | in   3028 run   2037 out   8000 | ld      0 cmp   2027 st      0 |  98.7% PASS
dscnn_pw2_50z DMA GEMM 125x64x64 w0=50% | in      7 run  19945 out      0 | ld   4914 cmp   2027 st  13000 |  98.7% PASS
resnet8_fc9   CPU GEMM 1x64x10          | in    180 run     46 out     10 | ld      0 cmp     39 st      0 |   6.4% PASS
resnet8_fc9   DMA GEMM 1x64x10          | in      7 run    354 out      0 | ld    286 cmp     39 st     20 |   6.4% PASS
CAPS register: GEMM yes, CONV yes
conv3x3       CPU CONV 8x8x4 k3 s1      | in    150 run    156 out    513 | ld      0 cmp    154 st      0 |  46.8% PASS
conv3x3       DMA CONV 8x8x4 k3 s1      | in     17 run   1223 out      1 | ld    226 cmp    154 st    832 |  46.8% PASS
dscnn_conv0   CPU CONV 49x10x1 k10 s2   | in    777 run   1520 out   8001 | ld      0 cmp   1518 st      0 |  82.3% PASS
dscnn_conv0   DMA CONV 49x10x1 k10 s2   | in     17 run  15765 out      1 | ld   1243 cmp   1518 st  13000 |  82.3% PASS
resnet8_conv3 CPU CONV 32x32x16 k3 s2   | in   5262 run   5205 out   8193 | ld      0 cmp   5195 st      0 |  88.7% PASS
resnet8_conv3 DMA CONV 32x32x16 k3 s2   | in     17 run  27040 out      1 | ld   8528 cmp   5195 st  13312 |  88.7% PASS
mbnet_dw1     CPU DW 48x48x8 k3 s1      | in   4640 run   1553 out  18433 | ld      0 cmp   1547 st      0 |  41.9% PASS
mbnet_dw1     DMA DW 48x48x8 k3 s1      | in     17 run  39030 out      1 | ld   7526 cmp   1547 st  29952 |  41.9% PASS
dscnn_dw1     CPU DW 25x5x64 k3 s1      | in   2158 run   1047 out   8001 | ld      0 cmp   1040 st      0 |  27.0% PASS
dscnn_dw1     DMA DW 25x5x64 k3 s1      | in     17 run  17525 out      1 | ld   3484 cmp   1040 st  13000 |  27.0% PASS
mbnet_pw4     CPU CONV 24x24x16 k1 s1   | in   2446 run   2334 out  18433 | ld      0 cmp   2326 st      0 |  49.5% PASS
mbnet_pw4     DMA CONV 24x24x16 k1 s1   | in     17 run  36236 out      1 | ld   3952 cmp   2326 st  29952 |  49.5% PASS
[accel] job rejected: does not fit the scratchpads
oversized job -> ERROR flag set (expected)
ALL TESTS PASSED (0 failures), simulated time 3582680 ns
[accel] SIGMA, 256 PEs, dn_bw 64, DMA 4 B/cycle, burst 64 B + 10 cycles
  …
resnet8_conv0 CPU GEMM 1024x27x16       | in   7024 run   8230 out  16384 | ld      0 cmp   8224 st      0 |  21.0% PASS
resnet8_conv0 DMA GEMM 1024x27x16       | in      7 run  46268 out      0 | ld  11410 cmp   8224 st  26624 |  21.0% PASS
dscnn_pw2     CPU GEMM 125x64x64        | in   3028 run   8263 out   8000 | ld      0 cmp   8257 st      0 |  24.2% PASS
dscnn_pw2     DMA GEMM 125x64x64        | in      7 run  26182 out      0 | ld   4914 cmp   8257 st  13000 |  24.2% PASS
dscnn_pw2_50z CPU GEMM 125x64x64 w0=50% | in   3028 run   4655 out   8000 | ld      0 cmp   4647 st      0 |  43.0% PASS
dscnn_pw2_50z DMA GEMM 125x64x64 w0=50% | in      7 run  22563 out      0 | ld   4914 cmp   4647 st  13000 |  43.0% PASS
  …
CAPS register: GEMM yes, CONV no
```

```
  …
conv3x3       CPU CONV on this architecture -> ERROR flag set (expected)
  …
conv3x3       DMA CONV on this architecture -> ERROR flag set (expected)
  …
ALL TESTS PASSED (0 failures), simulated time 1627300 ns
```

(Shortened: lines marked … are left out here; the full output is in `reference_results/unit_test_output.txt`.)

Each run ends with `ALL TESTS PASSED (0 failures)`. In each line, `in/run/out` are the driver's view (cycles to configure and copy in, START to DONE, copy out), and `dma_ld/compute/dma_st` are the accelerator's own counters. Two things to notice already: in DMA mode the store of the 32-bit outputs is the largest part of `run`; and the same layer takes very different compute cycles on the three fabrics.

> **If it fails**
>
> `undefined reference to sc_core::sc_api_version_2_3_3_cxx201703L`: something was compiled without `-std=c++20`; the Makefile sets it, so check you did not override `CXX_SC`. `STONNEModel.h: no such file`: STONNE is not at `/opt/soc-ext/stonne`; run 3.3 again or pass `make STONNE_DIR=...`. `getNCycles is not a member`: the patch was not applied (rerun 3.3).

5. Step B: first program on the platform

What it does. Compiles `sw/hello.c` for the RV32IM CPU, runs it on `accel32-vp`, prints a line, reads the accelerator's ID register over the bus and times a short loop with two different clocks.

container (bash)
```
make -C sw hello
accel32-vp --intercept-syscalls --quiet sw/hello.elf
```

you should see
```
[accel] TPU, 256 PEs, dn_bw 32, DMA 4 B/cycle, burst 64 B + 10 cycles
hello from RV32 on accel32-vp
accelerator ID register = 1398033989 (STNE, ok)
sum 0..999 = 499500 (ok)
loop: simulated time 30046 cycles, ISS rdcycle 17008 cycles
[accel32-vp] simulated time 352160 ns
```

> **Which clock to trust**
>
> `rdcycle` counts only instruction time inside the ISS. The platform timer (the accelerator's `NOW` registers, read by `now_cycles()`) is SystemC's simulated time, which also includes every bus transfer, memory access and peripheral delay. All measurements in this project use the platform timer. The CPU model is VP++'s default: every instruction fetch, load and store goes through the bus and a 1-cycle memory, so the CPU averages roughly 2.5–4 cycles per instruction. That is a slow microcontroller-class core; keep it fixed for all comparisons.

6. Step 0: reproduce the anchor paper's comparison (STONNE alone)

What it does. Runs STONNE's command-line simulator (no SystemC, no CPU) on eight representative MLPerf Tiny layer shapes with 256 multipliers each: the rigid TPU-like 16×16 mesh, MAERI (flexible, 256-wide distribution network, tile chosen by STONNE's mapper) and SIGMA (flexible, skips zeros) on dense and on 50%-zero weights. This repeats the kind of comparison made in the STONNE paper, on your workload. `ALL=1` runs every GEMM-shaped layer.

container (bash)
```
bash scripts/step0_reproduce_stonne.sh
```

layer	M	K	N	TPU 16×16	MAERI	SIGMA dense	SIGMA 50% zero weights	util TPU	util MAERI
resnet8_conv0	1024	27	16	3,968	2,061	4,118	2,058	43.5%	83.8%
resnet8_conv1	1024	144	16	11,456	9,236	16,591	6,225	80.4%	99.8%
resnet8_conv7	64	576	64	9,776	9,371	14,593	9,493	94.3%	98.3%
dscnn_conv0	125	40	64	2,388	1,271	1,520	946	52.3%	98.3%
dscnn_pw2	125	64	64	3,156	2,027	2,723	1,611	63.4%	98.7%
mbnet_pw2	2304	8	16	6,192	2,315	4,618	4,618	18.6%	49.8%
mbnet_pw14	36	128	128	3,816	2,379	3,246	1,855	60.4%	96.8%
mbnet_pw26	9	256	256	4,544	2,582	6,197	3,590	50.7%	89.2%

What to look for (and write about): the TPU mesh is near full use on large regular GEMMs (ResNet-8 conv7: 94%) but drops on shapes that do not fill a 16×16 grid (MobileNet pw2 with K = 8: 19%); MAERI with a full-width distribution network stays near 90–100% on almost every shape; SIGMA's cycles fall when weights are zero, the other two do not change. SIGMA's mapping depends on where the zeros are, so the same shape can give a different count with STONNE's random data here than with the benchmark's data in steps 2–4.

7. Step 1: software only

What it does. `sw/bench.c` loops over the 48 layers. For each layer it fills the input and weights with fixed pseudo-random INT8 values (same values in every run), runs the plain C kernel (`conv_sw`, `dwconv_sw` or `fc_sw`) and prints one CSV line with the cycles. The accelerator is present only for its timer.

container (bash)
```
bash scripts/step1_software.sh            # about 2–3 minutes
MODEL=dscnn bash scripts/step1_software.sh  # one model, faster
```

you should see
```
step1_sw.csv: 48 rows, 0 FAIL, 140s host time
[INFO,arch,TPU,pe,256,dn_bw,32,wzero,0,partition,gemm]
```

model	mode	total cycles	ms @100 MHz	speedup	im2col	move in	run	move out	CPU-only
ResNet-8	SW	383,321,507	3,833.22	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
DS-CNN	SW	106,535,249	1,065.35	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
MobileNet	SW	276,812,977	2,768.13	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%

> **Verify**
> The run line says `48 rows, 0 FAIL` and the log ends with `DONE` (`run_vp.sh` checks both and exits with an error otherwise). The totals must match the table exactly.

The CSV (`results/step1_sw.csv`) has one line per layer and mode: `model, layer, type, M, K, N, macs, mode, place, prep, in, run, out, total, acc_load, acc_compute, acc_store, check`; section 12.5 defines each column.

8. Step 2: a simple accelerator (rigid 16×16 TPU-like mesh)

The partition ("gemm"). Every standard convolution is lowered on the CPU with `im2col` into a GEMM $A[M\times K] \times B[N\times K]^T$, where M = output pixels, K = R·S·C (one filter) and N = filters. Pointwise and FC layers are already GEMMs. The accelerator runs the GEMMs. Depthwise layers have no GEMM form here and stay on the CPU.

The driver (`sw/accel_driver.c`): writes M, K, N and either copies A and B into the scratchpads (CPU-copy) or writes their memory addresses (DMA); writes `CTRL = START`; polls `STATUS` until DONE; reads C back (CPU-copy). Each layer runs in three modes, SW, CPU and DMA, and each accelerated result is compared with the SW result.

container (bash)
```
bash scripts/step2_simple_accel.sh        # about 4–6 minutes
```

model	mode	total cycles	ms @100 MHz	speedup	im2col	move in	run	move out	CPU-only
ResNet-8	SW	383,321,507	3,833.22	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	15,449,771	154.50	24.8x	72.9%	16.7%	0.4%	10.0%	0.0%
	DMA	11,698,611	116.99	32.8x	96.3%	0.0%	3.7%	0.0%	0.0%
DS-CNN	SW	106,535,249	1,065.35	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	18,047,699	180.48	5.9x	2.3%	1.4%	0.1%	4.0%	92.2%
	DMA	17,159,591	171.60	6.2x	2.4%	0.0%	0.6%	0.0%	97.0%
MobileNet	SW	276,812,977	2,768.13	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	53,347,342	533.47	5.2x	5.1%	2.9%	0.1%	4.8%	87.1%
	DMA	49,581,882	495.82	5.6x	5.5%	0.0%	0.9%	0.0%	93.7%

> **Verify**
>
> `step2_tpu.csv: 144 rows, 0 FAIL`. All accelerated layers PASS, so the hardware path computes the same numbers as the software. Totals must match the table.

Reading the result

- **ResNet-8** (33× faster with DMA): the accelerator's compute is now a few percent of the time; **im2col on the CPU takes 96%** of the remaining time.
- **DS-CNN and MobileNet**: the depthwise layers left on the CPU take **97%** (DS-CNN) and **94%** (MobileNet) of the time, which caps the speedup at 6× and 6×.
- **CPU-copy versus DMA**: copying operands word by word through the CPU costs more than the DMA in every model; the per-layer table shows the gap is largest for layers with big outputs (32-bit results).
- Tiny layers do not pay off: the FC layers (M = 1) use 3–8% of the mesh.

▼ Per-layer view of ResNet-8 (from `summarize.py --layers`)

layer	type	M	K	N	SW cycles	CPU cycles	DMA cycles	compute	util
resnet8_conv0	CONV	1024	27	16	22,436,391	1,615,256	1,231,944	3,968	43.5%
resnet8_conv1	CONV	1024	144	16	75,735,143	4,287,330	3,405,878	11,456	80.4%
resnet8_conv2	CONV	1024	144	16	75,735,143	4,287,330	3,405,878	11,456	80.4%
resnet8_conv3	CONV	256	144	32	37,853,383	1,167,237	863,237	5,728	80.4%
resnet8_conv4	CONV	256	288	32	67,770,119	1,983,970	1,510,196	10,336	89.2%
resnet8_conv5	CONV	256	16	32	5,438,471	281,326	128,234	1,632	31.4%
resnet8_conv6	CONV	64	288	64	33,881,623	608,605	390,539	5,168	89.2%
resnet8_conv7	CONV	64	576	64	59,836,055	1,074,786	705,812	9,776	94.3%
resnet8_conv8	CONV	64	32	64	4,616,343	140,110	56,190	1,072	47.8%
resnet8_fc9	FC	1	64	10	18,836	3,821	703	77	3.2%

9. Step 3: a better accelerator

Step 3 improves the system in two separate moves, so the paper can say which one matters.

3a. Better compute fabric, same partition

MAERI with 256 multipliers and a 256-wide distribution network replaces the 16×16 mesh; the CPU still does im2col and the depthwise layers. STONNE's mapper picks the tile of every GEMM.

3b. Move more work into hardware ("conv" partition)

The accelerator gets a second job type, `OP_CONV`: it reads the NHWC input and the TFLite weights directly, adds the zero padding itself, and runs standard and depthwise convolutions with STONNE's own convolution model. The CPU does no im2col, and the depthwise layers leave the CPU. Pointwise and FC layers still use `OP_GEMM` (they need no im2col). Only MAERI accepts `OP_CONV`; that is the practical meaning of a "flexible" fabric here.

container (bash)
```
bash scripts/step3_better_accel.sh        # about 8–12 minutes
```

3a: MAERI, same partition as step 2

model	mode	total cycles	ms @100 MHz	speedup	im2col	move in	run	move out	CPU-only
ResNet-8	SW	383,321,507	3,833.22	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	15,438,671	154.39	24.8x	73.0%	16.7%	0.3%	10.0%	0.0%
	DMA	11,687,511	116.88	32.8x	96.4%	0.0%	3.6%	0.0%	0.0%
DS-CNN	SW	106,535,249	1,065.35	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	18,042,019	180.42	5.9x	2.3%	1.4%	0.1%	4.0%	92.2%
	DMA	17,153,911	171.54	6.2x	2.4%	0.0%	0.6%	0.0%	97.0%
MobileNet	SW	276,812,977	2,768.13	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	53,318,822	533.19	5.2x	5.1%	2.9%	0.1%	4.8%	87.1%
	DMA	49,553,362	495.53	5.6x	5.5%	0.0%	0.8%	0.0%	93.7%

3b: MAERI with native convolution

model	mode	total cycles	ms @100 MHz	speedup	im2col	move in	run	move out	CPU-only
ResNet-8	SW	383,321,507	3,833.22	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	2,401,212	24.01	159.6x	0.0%	32.9%	2.6%	64.5%	0.0%
	DMA	275,915	2.76	1389.3x	0.0%	1.2%	98.6%	0.1%	0.0%
DS-CNN	SW	106,535,249	1,065.35	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	1,704,594	17.05	62.5x	0.0%	23.1%	0.8%	76.1%	0.0%
	DMA	169,883	1.70	627.1x	0.2%	1.6%	98.1%	0.2%	0.0%
MobileNet	SW	276,812,977	2,768.13	1.0x	0.0%	0.0%	0.0%	0.0%	100.0%
	CPU	6,315,299	63.15	43.8x	0.0%	33.3%	0.7%	66.0%	0.0%
	DMA	615,968	6.16	449.4x	0.1%	1.2%	98.5%	0.1%	0.0%

> **Verify**
> Both runs: `144 rows, 0 FAIL`.

Reading the result

- **3a changes almost nothing end-to-end** (11,687,511 vs 11,698,611 cycles for ResNet-8 with DMA) even though MAERI computes the GEMMs faster: the time is in im2col and depthwise layers on the CPU, not in the GEMMs. This is Amdahl's law, measured.
- **3b removes the CPU work**: ResNet-8 drops to 275,915 cycles (1389× over software), DS-CNN to 169,883 (627×) and MobileNet to 615,968 (449×). What remains is mostly data movement and compute on the accelerator.
- In 3b, CPU-copy mode is far slower than DMA: once the CPU stops computing, moving the operands through it becomes the bottleneck.

The absolute speedups over software are large because the software baseline is plain C on a slow core (section 15). The comparisons *between* steps 2, 3a and 3b use the same CPU, bus and data, and are the results to emphasize.

10. Step 4: design-space exploration and figures

What it does. `scripts/dse.py` builds two fast variants of the benchmark (DMA mode only, no CPU reference, `check = SKIP`) and runs them on many hardware configurations in parallel. Run it only after steps 2 and 3 passed, because those runs are the functional check for these configurations.

container (bash)
```
python3 scripts/dse.py --list              # what will run
python3 scripts/dse.py --jobs 8            # all sweeps, 8 at a time (about 10–20 min)
python3 scripts/dse.py --only maeri_bw     # one sweep
python3 scripts/plot_results.py            # figures in results/figures/
```

sweep	what changes	question it answers
`tpu_size`	mesh 8×8, 16×16, 32×32	does a bigger rigid array help, given the CPU work?
`maeri_bw`	MAERI distribution bandwidth 32 … 256	how much bandwidth does the flexible fabric need to beat the mesh?
`partition`	"conv" partition at bandwidth 64 and 256	how much of the gain comes from moving work vs. from the fabric?
`dma_width`	DMA 4, 8, 16 bytes per cycle	when does data movement become the limit?
`sparsity`	0, 50, 80% zero weights, SIGMA vs TPU	when are pruned weights worth a sparse fabric?

Total cycles per model for every run (DMA mode), from `results/dse_summary.csv`:

sweep	run	ResNet-8	DS-CNN	MobileNet
tpu_size	tpu8×8	11,858,850	17,194,520	49,689,228
tpu_size	tpu16×16	11,698,510	17,159,510	49,581,648
tpu_size	tpu32×32	11,662,470	17,149,530	49,551,548
maeri_bw	maeri256_bw32	11,976,300	17,182,190	49,632,278
maeri_bw	maeri256_bw64	11,835,820	17,181,840	49,601,578
maeri_bw	maeri256_bw128	11,736,910	17,163,190	49,579,298
maeri_bw	maeri256_bw256	11,687,410	17,153,830	49,553,128
partition	maeri256_bw64_conv	362,115	199,059	668,955
partition	maeri256_bw256_conv	275,915	169,799	615,695
dma_width	tpu16_dma4	11,698,510	17,159,510	49,581,648
dma_width	maeri256_bw256_conv_dma4	275,915	169,799	615,695
dma_width	tpu16_dma8	11,583,980	17,132,420	49,466,708
dma_width	maeri256_bw256_conv_dma8	211,065	122,989	441,725
dma_width	tpu16_dma16	11,526,740	17,118,890	49,409,188
dma_width	maeri256_bw256_conv_dma16	178,635	99,589	354,795
sparsity	sigma256_bw256_wz0	11,727,070	17,161,600	49,571,638
sparsity	tpu16_wz0	11,698,510	17,159,510	49,581,648
sparsity	sigma256_bw256_wz50	11,680,910	17,151,980	49,555,398
sparsity	tpu16_wz50	11,698,510	17,159,510	49,581,648
sparsity	sigma256_bw256_wz80	11,652,490	17,148,750	49,544,628
sparsity	tpu16_wz80	11,698,510	17,159,510	49,581,648

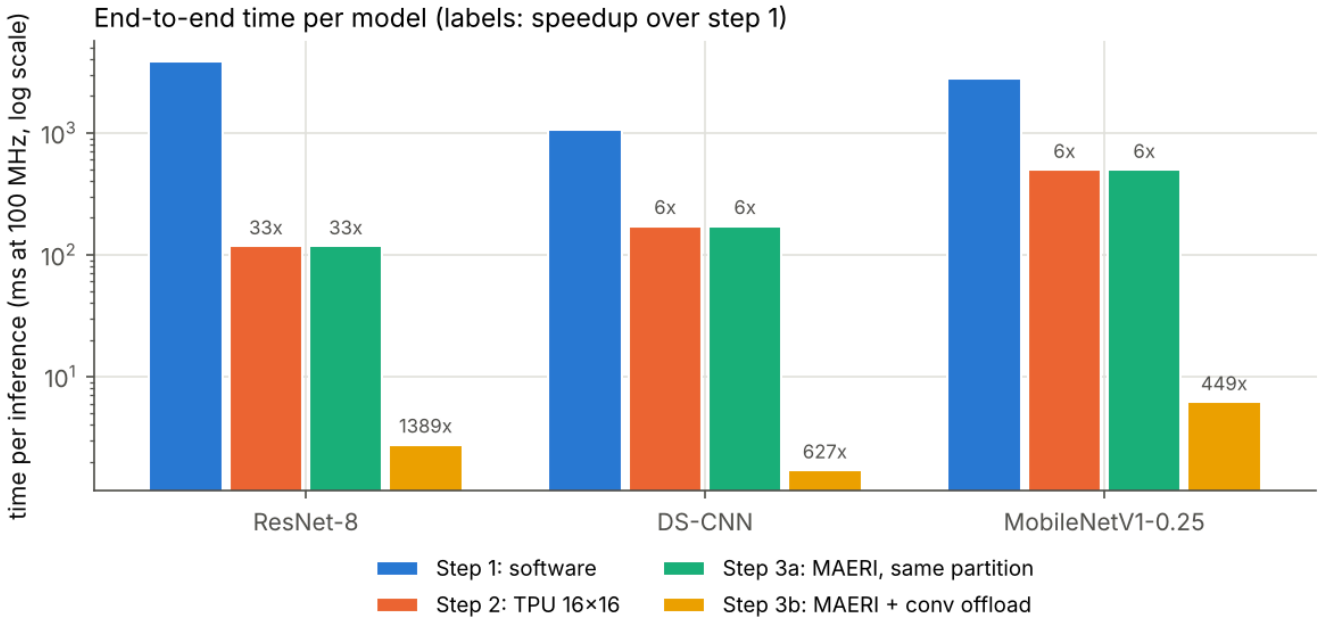


Figure 3. End-to-end time per model for steps 1–3 (DMA mode), log scale; labels are speedups over step 1.

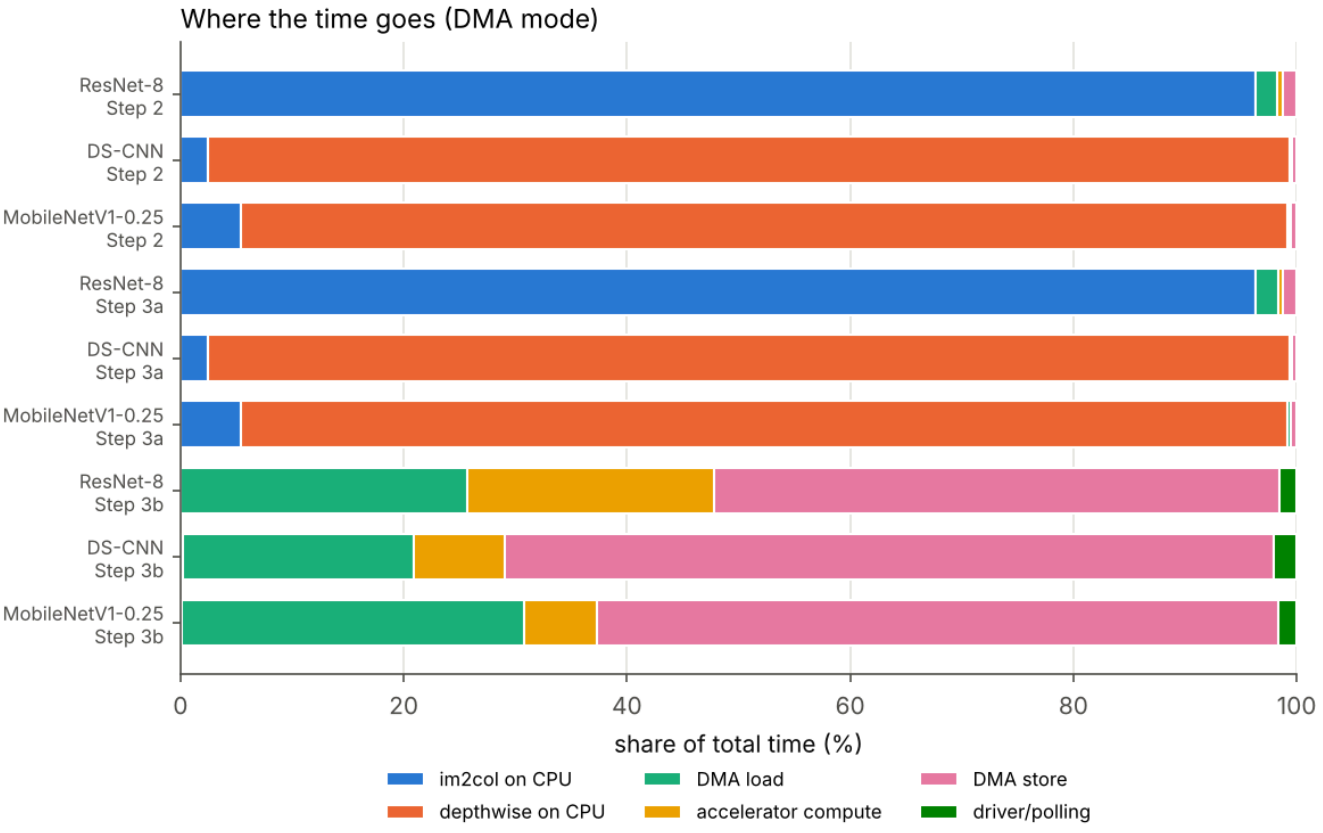


Figure 4. Where the time goes in each design (DMA mode).

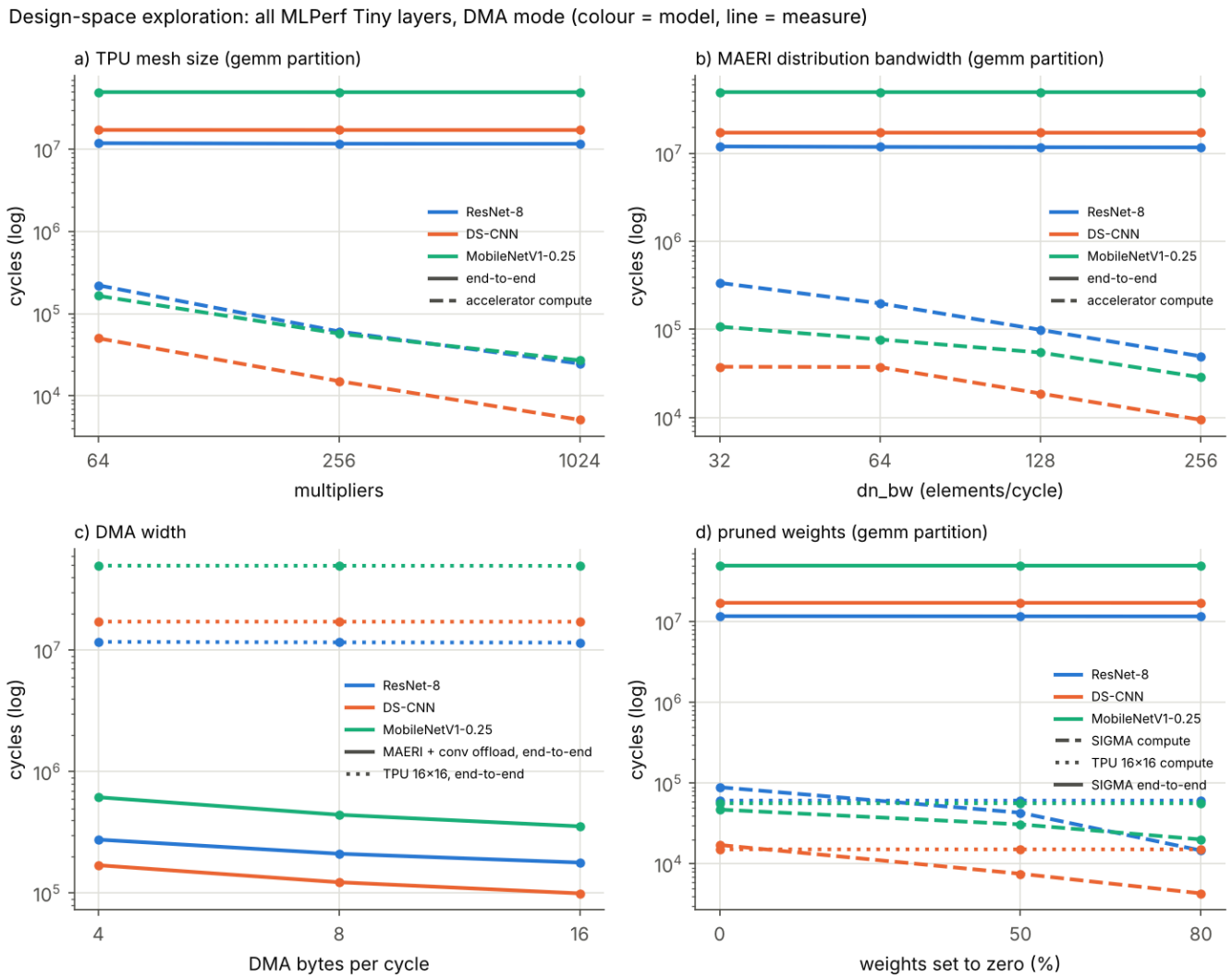


Figure 5. The four sweeps. Colour = model; line style = what is measured (see each legend): end-to-end cycles of all layers, or only the accelerator's compute cycles. Log scale.

Reading the sweeps (reference numbers, all models, DMA mode)

- **(a) Bigger rigid mesh.** Going from 8×8 to 32×32 multipliers cuts ResNet-8's accelerator compute from 221,008 to 24,633 cycles (9×), but its end-to-end time only from 11,858,850 to 11,662,470 cycles (1.7%). With the "gemm" partition, more multipliers buy almost nothing.
- **(b) Flexible fabric needs bandwidth.** MAERI's compute falls almost in proportion to its distribution bandwidth (ResNet-8: 338,464 cycles at 32 elements/cycle, 49,590 at 256). It needs the full 256-wide network to beat the 16×16 mesh on all three models (mesh: 60,669 / 15,091 / 57,101 compute cycles; MAERI at 256: 49,590 / 9,418 / 28,607). End-to-end, again, nothing changes.
- **Partition matters most.** With native convolution the end-to-end time drops from about 11.7 M to 0.28 M cycles for ResNet-8; once the CPU work is gone the fabric matters again (bandwidth 64 vs 256: 362,115 vs 275,915 cycles, 24% apart).
- **(c) Data movement is the next limit.** In the "conv" partition, a 4× wider DMA (4 → 16 bytes/cycle) cuts the end-to-end time by 35–42% (ResNet-8 275,915 → 178,635; DS-CNN 169,799 → 99,589; MobileNet 615,695 → 354,795). For the step-2 design the same change gives under 2%.
- **(d) Pruned weights.** SIGMA's compute drops with zero weights (ResNet-8: 89,228 → 43,090 → 14,665 cycles at 0, 50, 80% zeros) while the mesh stays at 60,669. Dense SIGMA is slower than the mesh on ResNet-8 (89,228 vs 60,669) but faster on MobileNet (47,111 vs 57,101), so the break-even point depends on the model (about 30% zero weights for ResNet-8). In the "gemm" partition the end-to-end effect is below 1%.

The common thread for the paper: **each accelerator improvement only shows up end-to-end once the part of the system that limits it is fixed first** (CPU work, then data movement).

11. Step 5: DMA and bus design (extension)

Why this step. After step 3b the CPU does almost nothing, and the accelerator spends most of its time moving data: the DMA load and store are 78% of the accelerator time for ResNet-8, 92% for DS-CNN and 93% for MobileNet. Steps 2–4 also made three convenient assumptions: the DMA never overlaps the computation, the CPU polls `STATUS` until the job is done, and the bus is ideal (the CPU and the DMA can use it at the same moment). Step 5 removes these assumptions one at a time, using the course's SoC topics: DMA, interrupts versus polling, and bus ownership and arbitration. It answers the fourth research question:

> **RQ4.** Once the computation is offloaded, how much do (a) overlapping transfers with compute (tiling and double buffering), (b) a completion interrupt instead of polling, and (c) the DMA's bus-ownership mode (burst/block, cycle stealing, transparent) and burst size change the end-to-end time and the CPU time left for other work?

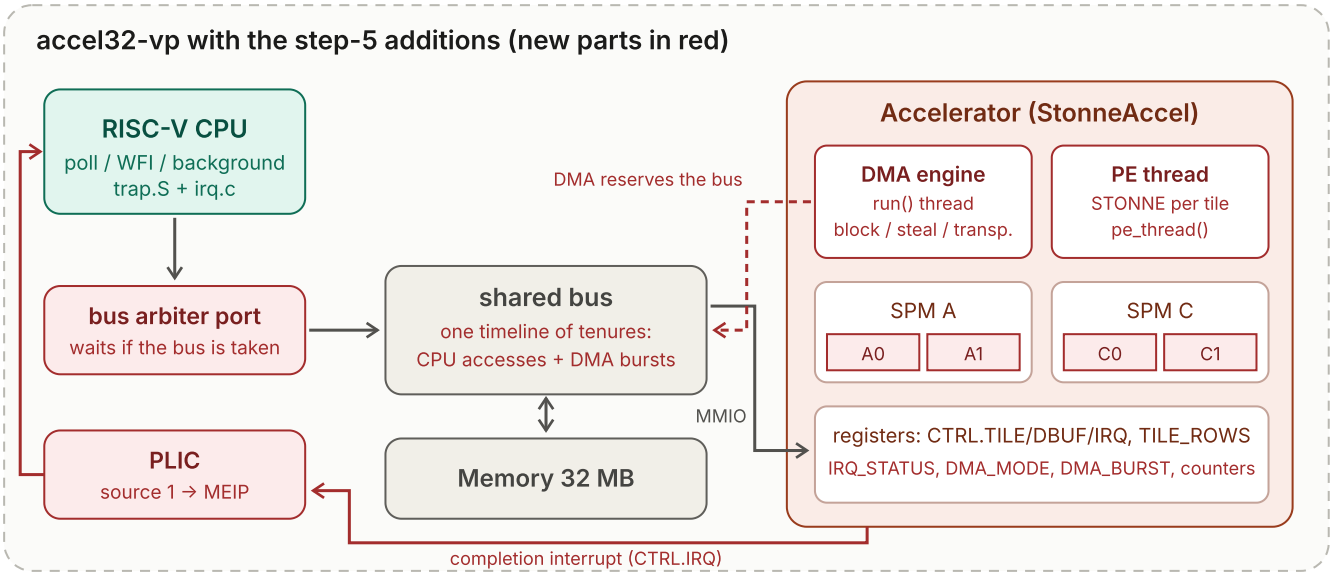
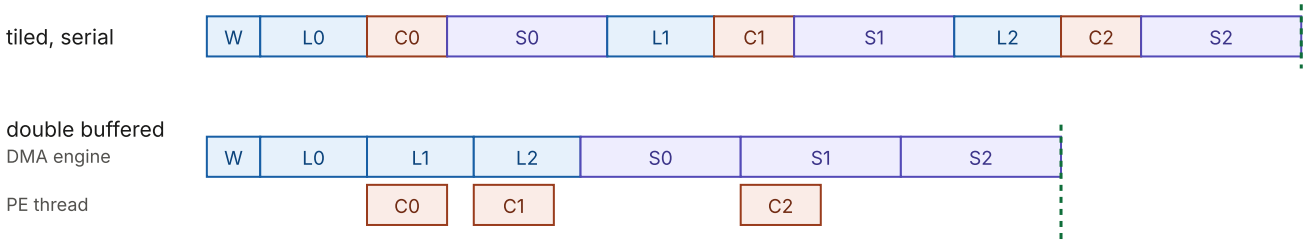


Figure 6. The step-5 additions. The CPU's bus port gets an arbiter, and the DMA reserves the same bus, so the two really compete. The accelerator has a separate DMA thread and PE thread, plus ping-pong halves of scratchpads A and C. Its completion interrupt goes through a PLIC to the core's machine external interrupt.



W = weights (once), L = load the inputs of a tile, C = compute it, S = store its 32-bit outputs.
There is one DMA engine, so loads and stores still follow each other: double buffering can only hide the compute.

Figure 7. One job cut into three tiles, without and with double buffering (to scale for a layer whose output store is twice its input load). The weights are loaded once; with double buffering the DMA loads tile i+1 and stores tile i−1 while tile i is computed.

All three changes are **off unless the software asks for them** (new `CTRL` bits, new registers, a new platform option). The programs of steps 1–4 do not use them, so steps 1–4 give exactly the numbers of sections 7–10, and the test of step 5a checks that automatically. The accelerator is the one from step 3b: MAERI, 256 multipliers, a 256-wide network, the "conv" partition and DMA mode.

11.1 Adding step 5 to your existing setup

The container you already built for steps 0–4 only needs the new files and a rebuild of the platform. Nothing new is downloaded.

1. Extract the new `soc-accel-project.zip` into the same Windows folder as before (`C:\kfupm\third semester\soc\Docker\systemc_work`) and let Windows **replace the files in the destination**. Your `results/` folder is kept.
2. In the container:

PowerShell (Windows)
```
docker start -ai soc-accel          # or a container made from soc-accel-ready
```

container (bash)
```
cd /workspace/soc-accel-project
source setup/env.sh
bash setup/04_build_vp.sh          # rebuilds accel32-vp with the step-5 platform (about 1 minute)
cd unit_test
make -j"$(nproc)"                  # builds tb_accel and the new tb_dma
make test                          # step A again: same output as in section 4
make test5                         # step 5-A, below
cd ..
```

you should see
```
built /opt/soc-ext/riscv-vp-plusplus/vp/build/bin/accel32-vp
...
04_build_vp: OK
```

After this, run `docker commit soc-accel soc-accel-ready` again (PowerShell) so the saved image has the new platform.

11.2 Step 5-A: test the extension alone (course SystemC 2.3.3)

What it does. `unit_test/tb_dma.cpp` builds a small SoC with the same accelerator source. A driver module plays the CPU and is connected through the bus arbiter port. A router serves memory and the accelerator, and the DMA reserves the same `SharedBus`. The accelerator's interrupt line goes into the driver. The test has three parts:

- **Part 1, tiling and double buffering.** Every GEMM and CONV job runs three ways: one piece, tiled, and tiled with double buffering. All three results must equal plain C++. Timing checks: a tiled job takes exactly load + compute + store; a double-buffered job is faster than the tiled one, but not faster than max(load + store, compute), because there is one DMA engine and one PE array. A double-buffered tile that does not fit half a scratchpad must be refused.
- **Part 2, interrupt.** A polling job counts its `STATUS` reads. An interrupt job must make zero polls and raise exactly one interrupt, and the line must go low after the acknowledge.
- **Part 3, bus ownership.** The same DMA-heavy job runs in each DMA mode, once with the CPU asleep and once with the CPU reading memory in a loop.

container (bash)
```
cd unit_test && make test5 && cd ..      # about 2–3 minutes
```

you should see
```
[accel] MAERI, 256 PEs, dn_bw 256, DMA 4 B/cycle, burst 64 B + 10 cycles
ID register: 0x53544e45, CAPS 0x1f (tile yes, irq yes, shared bus yes)
Part 1: tiling and double buffering (4 tiles), cycles START->DONE;
        tiled = load + compute + store; dbuf -x% = that much faster than tiled
resnet8_conv0 GEMM 1024x27x16      | one  40096 | tiled x4   40135 (= 11411+ 2100+ 26624) | dbuf  38035 - 5.2% | PASS
dscnn_pw2     GEMM 125x64x64       | one  19942 | tiled x4   20023 (=  4915+ 2108+ 13000) | dbuf  17915 -10.5% | PASS
mbnet_pw13    GEMM 9x256x256       | one  33887 | tiled x3   34443 (= 27561+ 3138+  3744) | dbuf  32039 - 7.0% | PASS
```

```
conv3x3       CONV 8x8x4 k3x3 s1   | one   1213 | tiled x4    1379 (=   315+  232+   832) | dbuf   1153 -16.4% | PASS
dscnn_conv0   CONV 49x10x1 k10x4 s2| one  15762 | tiled x4   16301 (=  1354+ 1947+ 13000) | dbuf  14787 - 9.3% | PASS
resnet8_conv3 CONV 32x32x16 k3x3 s2| one  27036 | tiled x4   27885 (=  9153+ 5420+ 13312) | dbuf  22465 -19.4% | PASS
mbnet_dw1     DW 48x48x8 k3x3 s1   | one  39026 | tiled x4   39995 (=  8463+ 1580+ 29952) | dbuf  38415 - 4.0% | PASS
dscnn_dw1     DW 25x5x64 k3x3 s1   | one  17525 | tiled x4   18425 (=  4265+ 1160+ 13000) | dbuf  17265 - 6.3% | PASS
tile sweep, resnet8_conv3 CONV 32x32x16 k3x3 s2:  2 tiles 22049  4 tiles 22465  8 tiles 23297  16 tiles 24961
[accel] job rejected: a tile does not fit half a scratchpad (lower TILE_ROWS)
dbuf with one 256 KB output tile -> ERROR flag set (expected)
same job with 4 tiles -> PASS
Part 2: completion interrupt
polling job: 1016 STATUS polls, CPU saw DONE 19946 cycles after START   PASS
interrupt job (dbuf): 0 polls, 1 interrupt(s), woke up 17919 cycles after START   PASS
Part 3: bus ownership. Job dma_heavy GEMM 1024x64x16, one piece; the CPU either sleeps (WFI) or keeps
reading memory (1 read + 1 cycle of work per loop) until the interrupt.
DMA mode              CPU    |   job cyc  DMA wait | CPU reads   CPU txn  CPU wait  max wait | result
block (burst 64 B)    sleep  |     57776         1 |         0         0         0         0 | PASS
block (burst 64 B)    busy   |     57777         2 |      2058      2058     53662     26624 | PASS
steal (burst 64 B)    sleep  |     57776         1 |         0         0         0         0 | PASS
steal (burst 64 B)    busy   |     59838      2063 |      4119      4119     51601        26 | PASS
steal (burst 4 B)     sleep  |    367376         1 |         0         0         0         0 | PASS
steal (burst 4 B)     busy   |    400398     33023 |     35079     35079    330241        11 | PASS
transparent           sleep  |     57776         1 |         0         0         0         0 | PASS
transparent           busy   |    111439     53664 |     55719     55719         0         0 | PASS
ALL TESTS PASSED (0 failures), simulated time 20375790 ns
```

(MAERI run; the TPU run that follows repeats parts 1–3 on GEMM jobs only. Full output: `reference_results/unit_test5_output.txt`.)

> **Verify**
>
> Both runs end with `ALL TESTS PASSED (0 failures)`. Every failed check prints a `CHECK FAILED:` line that says which property broke.

Part 3 is the lecture table on bus ownership, measured. With the CPU asleep, all modes take the same time; the one DMA wait cycle is the CPU's own START write. With a busy CPU:

- **Block mode** keeps the DMA at full speed, but the CPU waits up to a whole transfer (26,624 cycles here) and gets only about 2,000 memory reads done.
- **Cycle stealing** limits the CPU's longest wait to one burst (26 cycles with 64-byte bursts, 11 with 4-byte bursts). The cost is a slower DMA, and with small bursts a much slower one, because every burst pays the fixed latency.
- **Transparent** mode never delays the CPU, which does 27× more reads than in block mode, but the DMA job takes about twice as long.

11.3 Steps 5a–5c on the platform

What it does. `scripts/step5_dma_extension.sh` builds three variants of `sw/bench_dma.c` and runs every layer in three modes. **DMA** is the whole layer as one job, exactly as in step 3b. **TILE** cuts the job into about 4 tiles, which still run one phase after another. **DBUF** uses the same tiles with double buffering. The variants differ in how the CPU waits:

run	bus	CPU while the accelerator works	checked	output
5a	ideal	polls `STATUS` (as in steps 2–4)	yes, and DMA rows = step 3b	`results/step5a_dbuf.csv`
5b	ideal	completion interrupt, sleeps in `WFI`	yes	`results/step5b_irq_wfi.csv`
5c	shared, cycle stealing, 64-byte bursts	completion interrupt, runs background work (a checksum over a buffer in memory, standing in for other application work)	yes	`results/step5c_bus_steal_bg.csv`

container (bash)

```
bash scripts/step5_dma_extension.sh                # 5a-5c, then the sweeps (about 30-45 min on 2
cores)
SKIP_SWEEPS=1 bash scripts/step5_dma_extension.sh   # 5a-5c only
MODEL=dscnn bash scripts/step5_dma_extension.sh     # 5a-5c for one model, no sweeps
python3 scripts/summarize5.py results/step5a_dbuf.csv --compare results/step3b_maeri_conv.csv
```

5a: double buffering (ideal bus, the CPU polls)

model	mode	total cycles	vs one piece	accelerator jobs	DMA load	compute	DMA store	bytes read
ResNet-8	DMA	275,915	1.00×	271,582	70,976	60,810	139,796	174,704
	TILE	284,822	0.97×	280,539	75,796	64,947	139,796	186,544
	DBUF	229,542	1.20×	225,267	75,796	64,947	139,796	186,544
DS-CNN	DMA	169,883	1.00×	166,020	35,173	13,825	117,022	86,570
	TILE	174,498	0.97×	170,483	38,403	15,058	117,022	94,490
	DBUF	159,898	1.06×	155,897	38,403	15,058	117,022	94,490
MobileNet	DMA	615,968	1.00×	605,038	189,046	39,708	376,284	465,264
	TILE	636,084	0.97×	624,759	204,700	43,775	376,284	503,760
	DBUF	594,404	1.04×	583,123	204,700	43,775	376,284	503,760

5b: completion interrupt and WFI (ideal bus), compared with 5a

model	mode	total cycles	vs polling	CPU free while waiting	DONE → CPU reacts	STATUS polls	CPU bus wait	longest CPU wait	DMA bus wait
ResNet-8	DMA	279,702	+1.4%	98%	186	0	0	0	0
	TILE	288,721	+1.4%	98%	186	0	0	0	0
	DBUF	233,449	+1.7%	98%	186	0	0	0	0
DS-CNN	DMA	173,868	+2.3%	97%	186	0	0	0	0
	TILE	178,401	+2.2%	97%	186	0	0	0	0
	DBUF	163,815	+2.4%	97%	186	0	0	0	0
MobileNet	DMA	627,070	+1.8%	98%	186	0	0	0	0
	TILE	646,987	+1.7%	98%	186	0	0	0	0
	DBUF	605,351	+1.8%	98%	186	0	0	0	0

5c: shared bus with cycle stealing, the CPU doing background work, compared with 5a

model	mode	total cycles	vs polling	CPU free while waiting	DONE → CPU reacts	STATUS polls	CPU bus wait	longest CPU wait	DMA bus wait
ResNet-8	DMA	295,953	+7.3%	25%	148	0	203,805	260	13,713
	TILE	304,404	+6.9%	25%	147	0	208,486	260	14,005
	DBUF	248,760	+8.4%	11%	148	0	208,462	260	14,045
DS-CNN	DMA	186,737	+9.9%	15%	147	0	147,185	260	9,981
	TILE	190,373	+9.1%	15%	148	0	150,339	260	10,254
	DBUF	177,153	+10.8%	9%	150	0	150,303	260	10,212
MobileNet	DMA	667,888	+8.4%	13%	148	0	546,620	728	36,692
	TILE	691,096	+8.6%	14%	148	0	561,804	728	37,735
	DBUF	648,610	+9.1%	9%	148	0	561,754	728	37,781

> **Verify**
>
> Each run: `144 rows, 0 FAIL`. After 5a: `DMA rows equal to results/step3b_maeri_conv.csv: 48/48 -> IDENTICAL`, which proves that the extension did not change the step-3b design. The script stops if this check fails. The script ends with `step5: OK`. Totals must match the tables.

Columns. *CPU free while waiting* = the share of the time between START and the CPU carrying on that the CPU could use for something else. In WFI it is the time asleep. In BG it is the background work actually done, counted in cycles of a CPU that has the bus to itself (the program measures one work unit at start-up), so it drops when the bus slows the CPU down. When the CPU polls it is 0. *DONE → CPU reacts* = cycles from the accelerator setting DONE until the poll loop exits or the interrupt handler starts. *Bus waits* are summed over all jobs; the longest single CPU wait is the maximum.

Reading 5a–5c

- **Double buffering helps exactly as much as there is compute to hide.** DBUF is 1.20× faster than the one-piece job for ResNet-8, but only 1.06× for DS-CNN and 1.04× for MobileNet. With one DMA engine, overlap can hide the compute and nothing else. Compute is 22% of ResNet-8's accelerator time but only 7–8% for the other two, whose time is almost all the store of 32-bit outputs.
- **Tiling alone costs about 3%.** TILE is slower than DMA in every model. Each tile is a separate STONNE job, so it pays its own pipeline fill (compute up 7% for ResNet-8), and CONV tiles re-read their halo rows (ResNet-8's DMA load time rises from 70,976 to 75,796 cycles). Double buffering has to win back this overhead before it gains anything.
- **On an ideal bus the interrupt is slightly slower than polling**, by 1.4–2.4%. The CPU reacts to DONE after about 186 cycles: PLIC trigger, trap entry, 16 register saves, claim, acknowledge. A polling loop reacts in about 33. What the interrupt buys is the CPU: 97–98% of the waiting time is free (asleep in WFI), and the 27,162 (ResNet-8) to 60,515 (MobileNet) `STATUS` reads of polling disappear.
- **Once the bus is shared, "other work" is not free.** In 5c the DMA steals the bus burst by burst. Every CPU access that meets a burst waits for it (at most 26 cycles), so the background work progresses at only 9–25% of its speed on a private bus. The DMA, in turn, waits for the CPU's accesses, and the accelerator jobs take 5–6% longer. The end-to-end time grows by 7–11%. Double buffering keeps the DMA even busier, which leaves the CPU less: 9–11% instead of 13–25%.

11.4 The step-5 sweeps and figure

`scripts/dse5.py` runs 28 configurations with fast program variants (no CPU reference, `check = SKIP`; 5a–5c are the checked runs) and writes `results/dse5_summary.csv`. `scripts/plot_step5.py` draws Figure 8.

container (bash)

```
python3 scripts/dse5.py --list        # what will run
python3 scripts/dse5.py --jobs 8      # all sweeps (on your laptop about 10 minutes)
python3 scripts/dse5.py --only bus    # one sweep
python3 scripts/plot_step5.py         # results/figures/fig4_step5.png
```

sweep	what changes	question
`overlap`	DMA, TILE, DBUF at DMA width 4, 8, 16 B/cycle	when does overlapping transfers with compute pay off?
`tiles`	double buffering with 2, 4, 8, 16 tiles (width 4 and 16)	how many tiles: more overlap against halo re-reads and per-tile overhead
`wait`	polling, IRQ + WFI, IRQ + background work, ideal bus	what does an interrupt cost, and what does it free?
`bus`	shared bus: block, cycle stealing, transparent × the three wait styles	which ownership mode suits which CPU behaviour?
`burst`	burst 4 … 1024 B, block vs cycle stealing, CPU busy	DMA efficiency against the longest CPU stall (real-time response)

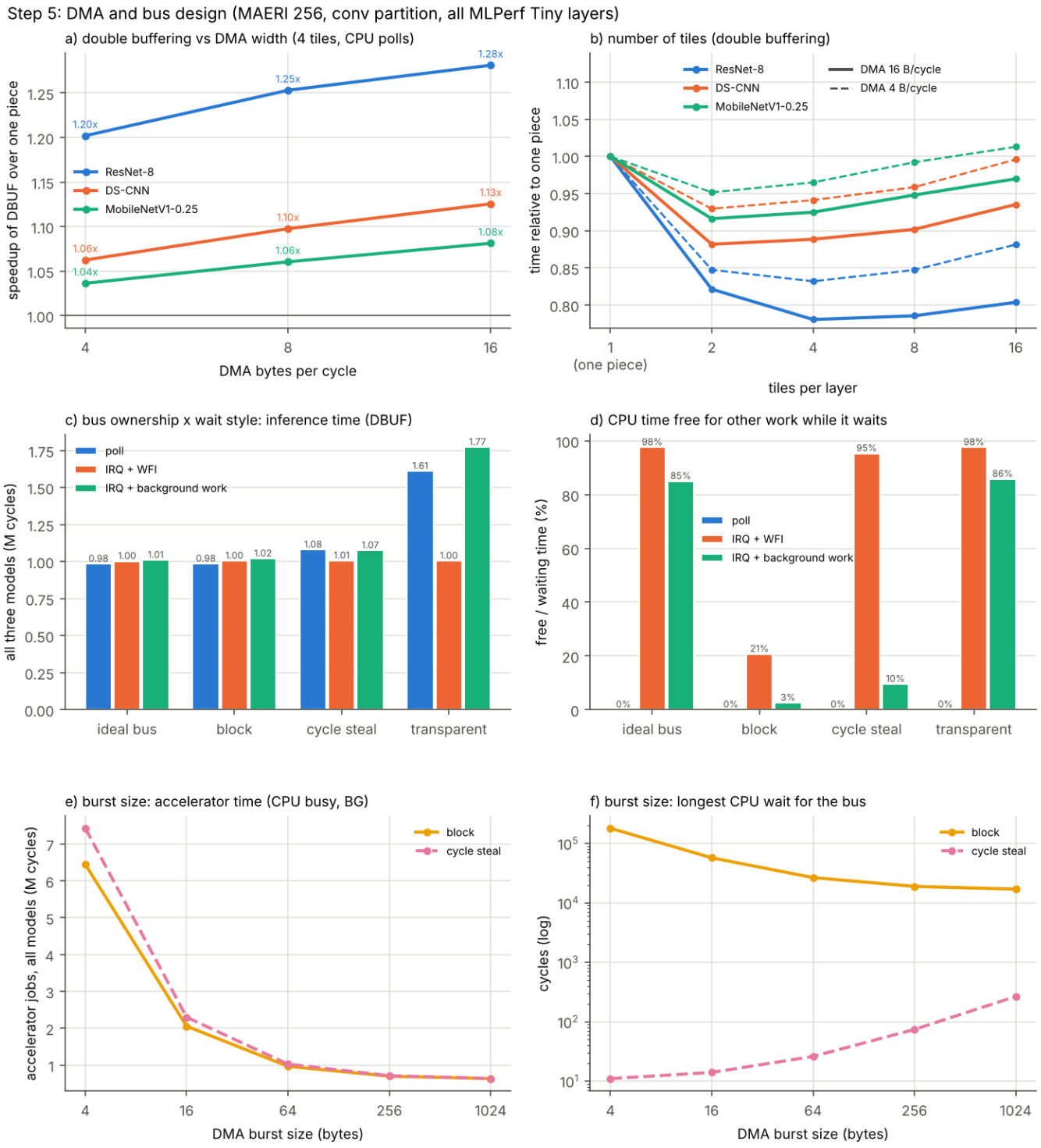


Figure 8. The step-5 sweeps, all MLPerf Tiny layers on the step-3b accelerator. (a) Speedup of double buffering over the one-piece job at three DMA widths. (b) Time against the number of tiles. (c, d) Inference time and the CPU time left free, for every combination of bus model and wait style. (e, f) Burst size: total accelerator time and the longest single CPU wait for the bus.

Bus ownership × wait style, double-buffered jobs, all three models together (from `results/dse5_summary.csv`):

bus	CPU waits by	total cycles (3 models)	vs ideal + polling	accelerator jobs	CPU free	CPU bus wait	longest CPU wait	DMA bus wait
ideal bus	polling	983,844	+0.0%	964,287	0%	0	0	0
	IRQ + WFI	1,002,615	+1.9%	964,287	98%	0	0	0
	IRQ + background	1,009,728	+2.6%	964,287	85%	0	0	0
block	polling	984,674	+0.1%	965,038	0%	951,740	26,624	773
	IRQ + WFI	1,004,239	+2.1%	964,588	21%	759,807	26,624	316
	IRQ + background	1,019,258	+3.6%	965,214	3%	951,774	26,624	947
cycle stealing	polling	1,079,494	+9.7%	1,059,886	0%	927,590	26	97,376
	IRQ + WFI	1,005,740	+2.2%	966,274	95%	25,829	26	1,989
	IRQ + background	1,074,523	+9.2%	1,025,227	10%	920,519	26	62,038
transparent	polling	1,610,424	+63.7%	1,590,801	0%	0	0	634,740
	IRQ + WFI	1,003,911	+2.0%	965,583	98%	0	0	1,296
	IRQ + background	1,771,104	+80.0%	1,723,586	86%	0	0	768,126

Reading the sweeps (the answers to RQ4)

- **Overlap pays more when the transfer gets faster** (Figure 8a). For ResNet-8, double buffering gives 1.20× at 4 B/cycle, 1.25× at 8 and 1.28× at 16. A wider DMA shrinks the transfer, so the compute that overlap hides becomes a larger share of the time. DS-CNN and MobileNet stay at 1.04–1.13× because their time is still mostly the 32-bit output store.
- **Few tiles are enough** (Figure 8b). The best is 4 tiles for ResNet-8 (0.78 of the one-piece time at 16 B/cycle) and 2 tiles for DS-CNN and MobileNet. Beyond that, the per-tile STONNE fill and the re-read halo rows cost more than the extra overlap saves. At 4 B/cycle with 16 tiles, MobileNet is even 1% slower than without tiling.
- **The interrupt only wins when the bus is shared** (Figure 8c, table). On the ideal bus polling is fastest, and the interrupt costs 1.9% (WFI) for 98% free CPU time. With cycle stealing, the polling CPU takes bus slots from the DMA (97,376 cycles of DMA waiting), and polling becomes 9.7% slower than on the ideal bus while IRQ + WFI stays at +2.2%. With transparent DMA, a polling CPU nearly starves the DMA (+63.7%), while WFI leaves it the whole bus (+2.0%).
- **Each ownership mode suits a different CPU** (Figure 8c, d):
  - **Block mode** keeps the accelerator at full speed whatever the CPU does (+0.1% with polling). It freezes a cacheless CPU for up to 26,624 cycles at a time, even on its way to `WFI`, so only 21% of the waiting time is really free, and background work gets 3%.
  - **Cycle stealing** bounds the longest CPU stall to one burst (26 cycles), but whoever is busier loses: background work gets 10% of the CPU.
  - **Transparent mode** gives the CPU 86% of its time for background work, but the inference becomes 80% slower. It is right for background transfers, not for the accelerator's critical path.
  - The best combination for this SoC is **IRQ + WFI** (about +2% in every mode), and the choice of ownership mode matters only when the CPU has other work to do.
- **The burst size is the biggest lever of all** (Figure 8e, f). Every burst pays a fixed 10-cycle latency: with 64-byte bursts that is 10 of every 26 cycles. Going from 64 to 1024-byte bursts cuts the accelerator time from 0.97 M to 0.63 M cycles (all models, CPU busy, block mode), more than double buffering gained. The price is the real-time response of the CPU. In block mode the longest CPU stall is a whole transfer

(17,024–180,224 cycles). With cycle stealing it is one burst: 11 cycles with 4-byte bursts and 266 with 1024-byte bursts. With 1024-byte bursts, cycle stealing is as fast as block mode (0.63 M) and its longest stall is 64× shorter. This is the lecture's point that the DMA's bus-ownership policy affects the whole SoC, measured.

11.5 How the step-5 code works

The shared bus (`accel/shared_bus.h`)

`SharedBus` keeps one timeline of bus tenures, as non-overlapping [start, end) intervals. `reserve(master, req, len)` returns the first free slot at or after `req` that is long enough. This is first come, first served, and a tenure already granted is never pre-empted. `BusArbiterPort` is a TLM pass-through module between the core and VP++'s `SimpleBus`. It forwards each transaction and takes the target's latency as the tenure length (memory 1 cycle, accelerator register 1 cycle, PLIC 4 cycles). It then reserves the bus at the CPU's local time and adds the wait to the transaction delay. The CPU is loosely timed (it may run up to one TLM quantum, here 1 cycle, ahead of SystemC time), and a timeline handles requests that arrive slightly out of order exactly. With the bus model off (the default), the arbiter is not even instantiated.

The DMA side is in `StonneAccel::dma_xfer()`. The data still moves through TLM exactly as before; only the timing depends on the mode:

- **block**: one reservation for the whole transfer, i.e. all its bursts of `latency + ⌈bytes/width⌉` cycles. The CPU is locked out until the transfer ends.
- **steal**: one reservation per burst. Between bursts the CPU's pending access gets the bus, so with `DMA_BURST = 4` this is classic one-word cycle stealing.
- **transparent**: the DMA reserves nothing. It waits for the time it still needs, then looks back at how much of that time the bus was really idle, and repeats until enough idle time has been collected. Looking back is exact, because no master can make a request in the past. The CPU never waits for the DMA.

The accelerator copies the bus statistics at START and DONE, so every job reports its own CPU transactions, CPU wait, longest CPU wait and DMA wait (registers 0xAC–0xB8, CSV columns `cpu_txn, cpu_wait, cpu_maxw, dma_wait`).

Tiles and double buffering (`accel/stonne_accel.h`)

`make_tiles()` cuts the job into tiles of `TILE_ROWS` rows: rows of A and C for a GEMM, output rows for a CONV. A CONV tile reads the input rows it needs, including the halo (the rows shared with the next tile, read twice) and its own top padding. With `CTRL_TILE` the control thread loads the weights once, then loads, computes and stores each tile in turn. With `CTRL_DBUF`, `run()` becomes the DMA engine and a second `SC_THREAD`, `pe_thread()`, computes. They synchronise through three counters (tiles loaded, computed, stored) and one `sc_event`:

- tile i may be computed when it is loaded and output buffer i mod 2 has been stored (`n_loaded ≥ i+1`, `n_stored ≥ i−1`);
- tile i+1 may be loaded into input buffer (i+1) mod 2 when tile i−1 has been computed (`n_computed ≥ i`);
- tile i−1 may be stored when it has been computed.

The two buffers are the two halves of scratchpads A and C. A tile that does not fit half a scratchpad is refused with `ERROR`. Each tile is a separate STONNE job, so its pipeline fill and drain are counted again; that is the per-tile overhead seen in TILE.

The interrupt path

With `CTRL_IRQ`, the accelerator raises its interrupt line when the job ends (also on an error) and keeps it high until software writes 1 to `IRQ_STATUS`. In `accel32-vp` the line drives source 1 of an FE310-style PLIC at 0x0C00_0000, which raises the core's machine external interrupt (MEIP). On the software side:

- `irq_init()` sets `mtvec` to `trap_entry` (in `trap.S`: save the 16 caller-saved registers, call the C handler, restore, `mret`), sets the PLIC priority, enable and threshold, and sets `mie.MEIE` and `mstatus.MIE`.
- The handler claims the interrupt at the PLIC, acknowledges it at the accelerator, records the time, sets `accel_irq_flag` and completes the claim.
- `irq_wait_wfi()` avoids the classic lost-wake-up race. It clears `mstatus.MIE`, checks the flag, executes `wfi`, and only then sets `MIE` again. `WFI` wakes on a pending enabled interrupt even while `MIE` = 0, so an interrupt that arrives between the check and the `wfi` is not lost; it is taken as soon as `MIE` is set.

New registers (offsets from 0x1000_0000)

offset	name	use	offset	name	use
0x04	CTRL bits 3–5	TILE, DBUF, IRQ (per job)	0x98	POLLS	STATUS reads while the last job was busy
0x88	TILE_ROWS	rows per tile	0x9C–0xA0	DONE_LO/HI	time the last job finished
0x8C	IRQ_STATUS	pending; write 1 = acknowledge	0xA4	DMA_MODE	0 block, 1 steal, 2 transparent
0x90	CYC_TOTAL	START → DONE of the last job	0xA8	DMA_BURST	burst size in bytes
0x94	TILES	tiles of the last job	0xAC–0xB8	BUS_*	last job: CPU txn, CPU wait, CPU max wait, DMA wait
`CAPS` gains bit 2 (tiles), bit 3 (interrupt) and bit 4 (shared-bus platform). `DMA_MODE` and `DMA_BURST` start at the platform's `--accel-dma-mode` / `--accel-dma-burst` values; software may change them, as on a real DMA controller.					

New platform options and program variants

option / target	meaning
`--accel-bus-model`	put the arbiter in front of the CPU and give the DMA the same bus (default off: ideal bus)
`--accel-dma-mode block|steal|transparent`	reset value of `DMA_MODE` (only matters with the bus model)
`make -C sw dma`	`bench_dma_poll.elf`: modes DMA, TILE, DBUF, the CPU polls; the DMA rows use the unchanged step-2–4 driver
`make -C sw dma WAIT=wfi` / `WAIT=bg`	completion interrupt with WFI / with background work
options	`NTILES=8`, `MODES=dbuf` (or `one`, `tile`, lists such as `one,dbuf`), `CHECK=0` (fast, no reference), `ONLY_MODEL=dscnn`, `PARTITION=gemm`

The step-5 CSV adds the columns `wait, tiles, acc_total, bytes_in, polls, sleep, bg_units, resp, cpu_txn, cpu_wait, cpu_maxw, dma_wait`. Their meaning is the same as the summary columns above, per layer.

11.6 Limits of the step-5 models (say these in the paper)

- **One shared bus, no split transactions.** A DMA burst holds the bus for its whole latency plus its data beats. A pipelined or split-transaction bus (AXI-like) would let other masters use the latency cycles, and a multi-layer bus or crossbar (lecture 3) would remove most of the contention.
- **No caches.** Every CPU instruction fetch goes over the bus (VP++'s default timing), so a CPU that is merely running code competes with the DMA. A core with an instruction cache or tightly coupled memory would compete much less; this is the worst case for the CPU.
- **Transparent mode is idealised.** The DMA uses every idle bus cycle, even single ones, and pays its burst latency in pieces. Real transparent DMA needs idle windows long enough for a transfer, so it would be slower than shown.
- **One DMA engine.** Loads and stores of different tiles never overlap each other; a second channel or a wider bus would.

- **Interrupt cost.** The cost comes from VP++'s instruction timing and the PLIC model (4-cycle register access, 1-cycle trigger delay); a real core adds pipeline and memory effects. The background work is a synthetic checksum loop, not real application code.

12. How the code works

12.1 The accelerator model (`accel/stonne_accel.h`)

`StonneAccel` is an `sc_module` with two TLM-2.0 sockets: a target socket for the CPU (registers and three scratchpads) and an initiator socket for its DMA. Hardware parameters (`AccelConfig`) are fixed at start-up, like an IP block's generics; software reads them from read-only registers.

- **Register accesses** (`b_transport`): decode the offset, read or write the register, and add `reg_latency` cycles (default 1) to the transaction delay. Writing `CTRL.START` notifies `start_ev` at the time the CPU's write happens.
- **Control thread** (`run`, an `SC_THREAD`): waits for `start_ev`, checks the job fits the scratchpads, then (DMA mode) loads the two inputs, calls STONNE, waits for the compute cycles, (DMA mode) stores the output, and sets `DONE`.
- **DMA timing model**: the data really moves through the bus (functional), and the time charged is, per burst of *b* bytes, `latency + ceil(b / width)` cycles, i.e. `cycles = Σ (dma_burst_latency + [bytes/dma_bytes_per_cycle])`. Defaults: 64-byte bursts, 10 cycles latency, 4 bytes/cycle.
- **Compute timing**: STONNE's cycle count for the job, at the same 100 MHz clock as the CPU.

Register map (`accel/accel_regs.h`, base 0x1000_0000)

offset	name	use	offset	name	use
0x00	ID	reads 0x53544E45 ("STNE")	0x30–0x38	CYC_LOAD/COMPUTE/STORE	last job's phase cycles
0x04	CTRL	START, USE_DMA, CLEAR_DONE	0x3C–0x40	BYTES_IN/OUT	bytes moved by the last job
0x08	STATUS	BUSY, DONE, ERROR	0x44–0x48	NOW_LO/HI	platform timer (cycles)
0x0C–0x14	M, K, N	GEMM shape	0x50	OP	0 = GEMM, 1 = CONV
0x18–0x20	A/B/C_ADDR	memory addresses (DMA)	0x54	CAPS	bit0 GEMM, bit1 CONV
0x24–0x2C	ARCH, NUM_PE, DN_BW	hardware parameters	0x58–0x84	CONV_*	H, W, C, R, S, KOUT, G, stride, pads, Ho, Wo
Scratchpads: A at +0x10_0000 (256 KB, int8), B at +0x20_0000 (128 KB, int8), C at +0x30_0000 (256 KB, int32).					

12.2 The STONNE backend (`accel/stonne_backend.cpp`)

STONNE's headers define many global names (`CONV`, `FC`, `LEFT`...), so they are included only in this file. It converts the INT8 operands to STONNE's `float` type (every partial sum is an integer below 2^{24}, so the result is exact), configures one of three architectures, runs STONNE, and converts the output back to int32.

arch	STONNE configuration	mapping
TPU	OS_MESH multiplier network, TPU_OS_DENSE memory controller, TEMPORALRN reduction, rows+cols read ports	tile = whole mesh (T_M = rows, T_N = cols, T_K = 1)
MAERI	LINEAR multiplier network, ASNETWORK reduction tree, MAERI_DENSE_WORKLOAD, accumulation buffer	GEMM: STONNE Mapper; CONV: mRNA; depthwise: one channel per R·S multipliers, as many channels and output columns side by side as fit
SIGMA	LINEAR, SIGMA_SPARSE_GEMM, bitmap-compressed operands	weights stationary (`MK_STR_KN_STA`), so zero weights are skipped

The depthwise tile matters: STONNE's mRNA mapper runs one channel at a time (9 of 256 multipliers busy). The hand-written tile in `stonne_conv()` made MobileNet's first depthwise layer about 12× faster in cycles. This is a mapping decision you can discuss.

12.3 The platform (`vp_platform/accel/main.cpp`)

It is VP++'s `tiny32-vp` platform plus the accelerator: one RV32IM core (ISS), 32 MB memory at 0, CLINT, the syscall device, and the accelerator at 0x1000_0000. The accelerator's DMA is the third bus master and passes through VP++'s `PeripheralWriteConnector` so it respects the CPU's atomic-operation bus lock.

option	default	meaning
`--accel-arch`	tpu	tpu | maeri | sigma
`--accel-rows`, `--accel-cols`	16, 16	TPU mesh size
`--accel-ms`	256	MAERI/SIGMA multipliers (power of 2)
`--accel-dnbw`, `--accel-rnbw`	64, = dnbw	MAERI/SIGMA distribution and reduction bandwidth
`--accel-dma-width`, `--accel-dma-burst`, `--accel-dma-latency`	4, 64, 10	DMA timing model
`--accel-reg-latency`	1	cycles per CPU access to the accelerator
`--accel-verbose`, `--accel-stats`	off	show STONNE's console output / write its JSON statistics files
`--accel-bus-model`, `--accel-dma-mode`	off, block	step 5: shared, arbitrated bus and the DMA's bus-ownership mode (section 11.5)

12.4 The software (`sw/`)

Bare-metal RV32IM, no C library: `bootstrap.S` calls `main` and exits through the VP's syscall interface; `runtime.c` prints with the `write` syscall and provides `memcpy` / `memset`. `kernels.c` holds the software layers and `im2col` (TFLite layouts: input [H][W][C], conv weights [K][R][S][C], depthwise weights [R][S][C]). Build variants are selected in `sw/Makefile`:

target	file	modes	used by
`make`	bench_sw.elf	SW	step 1
`make acc`	bench_acc.elf	SW, CPU, DMA ("gemm" partition)	steps 2, 3a
`make conv`	bench_conv.elf	SW, CPU, DMA ("conv" partition)	step 3b
`make dse`	bench_dse*.elf	DMA only, no reference	step 4
`make dma` (+ `WAIT=wfi|bg`)	bench_dma_*.elf	DMA, TILE, DBUF ("conv" partition), see 11.5	step 5
Options for any target: `ONLY_MODEL=dscnn` (one model), `WZERO=50` (50% zero weights).			

12.5 What each CSV column means

`M, K, N, macs`	GEMM view of the layer (for depthwise: M = output pixels, K = R·S, N = channels); macs = M·K·N
`mode`	SW (CPU only), CPU (accelerator, CPU copies data), DMA (accelerator, DMA moves data)
`place`	where the layer actually ran: CPU or ACC
`prep`	CPU cycles for im2col before the job ("gemm" partition, CONV layers only)
`in`	cycles from the first register write to START (includes the CPU copying A and B)
`run`	cycles from START until the CPU sees DONE (includes DMA load/store in DMA mode)
`out`	cycles for the CPU to read C back (CPU-copy mode)
`total`	prep + in + run + out (for SW or CPU-placed layers: the software time)
`acc_load, acc_compute, acc_store`	the accelerator's own counters: DMA load, STONNE compute, DMA store
`check`	PASS if the accelerator's output equals the CPU's; SKIP in DSE runs

Utilization of the multipliers = macs / (number of multipliers × acc_compute). For SIGMA with zero weights it can exceed 100% because macs counts the dense work.

12.6 Making changes

- **Different hardware parameters:** pass options to `run_vp.sh`, e.g. `bash scripts/run_vp.sh bench_acc.elf my.csv --accel-arch maeri --accel-dnbw 128`. No rebuild needed.
- **Edited anything in `accel/` or `vp_platform/`:** rebuild the platform with `bash setup/04_build_vp.sh` (incremental, under a minute), and rerun `cd unit_test && make test`.
- **Edited the software:** the step scripts call `make` themselves.
- **A new sweep:** add a block of `RUNS.append(...)` lines in `scripts/dse.py`.
- **Regenerate the layer table** from the models: `python3 workloads/extract_layers.py $TINY_DIR/benchmark/training sw/layers.h workloads/layers_mlperf_tiny.csv`.
- **A new step-5 sweep:** add `RUNS.append(...)` lines in `scripts/dse5.py`; build options of the step-5 program are in `sw/Makefile` (`WAIT`, `NTILES`, `MODES`, `CHECK`).
- **Ideas for further work** (the open questions left after step 5): requantize outputs to INT8 in the accelerator (the 32-bit output store dominates the DMA time even with double buffering); a second DMA channel so loads and stores overlap each other; an im2col address generator in the DMA for the TPU.

13. Verification checklist

check	how	expected
STONNE builds and is correct	`setup/03_build_stonne.sh`	three PASS lines with the cycle counts of 3.4
platform builds	`setup/04_build_vp.sh`	`04_build_vp: OK`
accelerator model correct in isolation	`unit_test: make test`	three times `ALL TESTS PASSED`
CPU, bus, printing, timer	hello.elf	"(STNE, ok)" and "(ok)"
software baseline	step 1	48 rows, 0 FAIL, totals of section 7
hardware path = software path	steps 2, 3a, 3b	144 rows, 0 FAIL each
numbers reproduce	compare with `reference_results/`	identical cycle counts
a quick diff	`diff results/step2_tpu.csv reference_results/step2_tpu.csv`	no output
step-5 hardware alone	`unit_test: make test5`	twice `ALL TESTS PASSED`
step-5 extension does not change steps 2–4	step 5a	`DMA rows equal to … 48/48 -> IDENTICAL`
tiles, double buffering, interrupts and the shared bus compute the right numbers	steps 5a, 5b, 5c	144 rows, 0 FAIL each

14. Troubleshooting

symptom	cause and fix
`docker` not recognized	open a new PowerShell window after installing Docker Desktop
`Permission denied` running a script or program in /workspace	Windows folders drop the execute bit: run scripts with `bash script.sh`; the Makefiles `chmod +x` their outputs
`accel32-vp: command not found`	`source setup/env.sh` (or run `setup/05_save_env.sh` once)
link error `sc_api_version_2_3_3_cxx...`	a SystemC 2.3.3 program compiled without `-std=c++20`
`ERROR: accelerator not found`	you ran a benchmark on `tiny32-vp` instead of `accel32-vp`
`ERROR: this program uses OP_CONV`	bench_conv.elf needs `--accel-arch maeri`
`program did not finish` from run_vp.sh	look at the `.log` next to the CSV; usually a STONNE assertion from an unsupported parameter (e.g. MAERI/SIGMA sizes must be powers of two)
`WARNING: Could not open file parameters/mRNA_energy_parameters.txt`	harmless; `run_vp.sh` and `make test` copy that folder into the run directory to avoid it
VP++ build fails in 3.5	read `/opt/soc-ext/riscv-vp-plusplus/vp/build/build.log`; most often a missing Boost package (rerun 3.2)
`ERROR: the accelerator has no step-5 features`	the platform was built from the old sources: rerun `bash setup/04_build_vp.sh`
`[accel] job rejected: a tile does not fit half a scratchpad`	only with your own driver code: lower `TILE_ROWS` (the step-5 driver picks a size that fits)
a step-5 run hangs (no `DONE`)	an interrupt was lost: check you kept the `mstatus.MIE` masking around `wfi` in `irq_wait_wfi()`, and that the handler acknowledges `IRQ_STATUS` before completing the PLIC claim
different cycle counts from the reference	a different commit of STONNE or VP++, a changed default in `run_vp.sh`, or edited sources; rerun 3.3 on a clean `/opt/soc-ext`

15. Limits of the model (say these in the paper)

- **Slow CPU baseline.** The software is plain C (no SIMD, no CMSIS-NN-style tricks) on VP++'s timing model (about 2.5–4 cycles per instruction). Absolute speedups over software are therefore large; compare designs with each other, all on the same CPU.
- **Simple DMA model (steps 2–4).** Per-burst latency plus bytes/width, on an ideal bus (no contention with the CPU), no overlap of DMA and compute. Step 5 adds overlap and a contention model; its own limits are in section 11.6.
- **Free compression and layout.** SIGMA's bitmap compression and the reordering of depthwise weights are done functionally without being charged cycles (in practice weights are prepared once, offline).
- **STONNE's timing only.** Compute cycles come from STONNE's cycle-level model, which its authors validated against MAERI's RTL; there is no synthesis here, so no area or power claims. STONNE has an energy table model (`--accel-stats` writes its counters) if you want relative energy.
- **Layer-level benchmark.** Each layer runs on fresh random data rather than the previous layer's output; pooling, residual additions and requantization are not included. Cycle counts do not depend on the values except for SIGMA (zero skipping).

16. References

1. F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE Computer Architecture Letters*, 20(2), 2021; extended version in IISWC 2021. github.com/stonne-simulator/stonne

2. V. Herdt, D. Große, P. Pieper, R. Drechsler, "RISC-V based virtual prototype: An extensible and configurable platform for the system-level," *Journal of Systems Architecture*, vol. 109, 2020.
3. M. Schlägl, C. Hazott, D. Große, "RISC-V VP++: Next generation open-source virtual prototype," OSDA 2024. github.com/ics-jku/riscv-vp-plusplus
4. C. Banbury et al., "MLPerf Tiny benchmark," NeurIPS Datasets and Benchmarks, 2021. github.com/mlcommons/tiny
5. SiFive, "FE310-G000 Manual," chapter on the Platform-Level Interrupt Controller (the PLIC model used in step 5); A. Waterman, K. Asanović (eds.), "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture" (mtvec, mie/mip, WFI).
6. COE 407/661 lecture notes: "SoC Anatomy" (DMA, bus ownership: burst, transparent, cycle stealing) and "System Modeling and Simulation" (CPU/DMA/RAM on a shared bus).
7. H. Kwon, A. Samajdar, T. Krishna, "MAERI," ASPLOS 2018. E. Qin et al., "SIGMA," HPCA 2020. N. P. Jouppi et al., "TPU," ISCA 2017.

The full reference list with venues is in the proposal (docs/).